

AXIOM Dev Agent v2 (four-layer constitutional code agent)

A reviewer-first agent that wraps proposed code changes in four governance layers before allowing them to ship. Lifts the developmental pattern from CPI (ORVL-022 / PRs #6–#9) into software engineering.

The v2 implementation lives in `axiom_dev_agent_v2.py:1`. The single-loop predecessor `axiom_dev_agent.py` is retained for training-data capture; v2 is the recommended entry point.

The four layers

Layer	Name	Role
0	Reflex	Pure-Python AST + forbidden-pattern checks. Sub-ms.
1	Reviewer	Per-task-class competence + PR survival forecast.
2	Curriculum	AXM-backed memory of past tasks. Persists competence.
3	Examiner	Sealed CI suite signed under an independent key.

Each layer can emit `PASS`, `SOFTEN`, or `VETO`. A `VETO` from any layer halts the task; a `SOFTEN` returns advisory text the developer must address before proceeding.

Reviewer signals (Layer 1)

The forecast combines three penalties:

- **Novelty penalty** — `0.50 × (1.0 - competence_for_task_class)`
- **Size penalty** — `min(0.30, diff_lines / 1000)`
- **Citation penalty** — `0.20` if `cited_patterns` is empty or doesn't match the AXM corpus, else `0.0`

Verdict thresholds:

- `min_pred ≥ min_safe` → **PASS**
- `min_pred ≥ FLOOR_PASSING_LIKELIHOOD` → **SOFTEN** with advice
- otherwise → **VETO**

Competence updates after each task: `+COMPETENCE_BUILD_PER_SUCCESS` on clean CI pass, `-COMPETENCE_DROP_ON_VETO` on review veto, `-COMPETENCE_DROP_ON_CI_FAIL` on CI failure (`on_outcome` at `axiom_dev_agent_v2.py:380`).

Curriculum (Layer 2)

Past task trajectories are persisted via AXM (`axiom_axm.AXMContainer`). Similarity over the AXM-derived embedding transfers competence between related task classes, so a developer who has shipped a clean `refactor_extract_function` task earns a partial competence boost on `refactor_inline_function` . The curriculum also suggests the next task in the zone of proximal development.

Examiner (Layer 3)

The Examiner runs a sealed CI suite signed under an independent derived key — the agent under test cannot read the certificate or forge a passing one. Mirrors the air-gap pattern from `axiom_motion_examiner.py` .

API

```
from axiom_dev_agent_v2 import DevAgentV2, DevTask

agent = DevAgentV2()
task = DevTask(
    id="task-001",
    task_class="add_constitutional_guard",
    proposed_diff=open("change.diff").read(),
    cited_patterns=["GUARD_PATTERN_PROMPT_INJECTION"],
)
verdict = agent.reviewer.review(task)
# verdict.verdict    - "PASS" / "SOFTEN" / "VETO"
# verdict.reasons    - tuple of human-readable reason strings
# verdict.advice     - softening advice (only when verdict == "SOFTEN")
# verdict.signature  - HMAC over the verdict body
```

Task classes

The reviewer tracks competence per class — see `TASK_CLASSES` in `axiom_dev_agent_v2.py` . Each class accumulates its own score; the forecast uses only that class's history. Add a new class by extending the tuple and seeding `_competence` via `PullRequestReviewer.set(...)` .

v1 vs v2

v1 (`axiom_dev_agent.py`) writes every interaction to `axiom_dev_training.jsonl` — the Mistral fine-tune dataset. v2 wraps the loop in governance but does not write training data. If you need both, run them in parallel against the same task.